1 - Overview

This document specifies the communication protocol between the two servers within the Amazon/UPS system. It governs all interactions directly between these two parties.

Amazon operates the online store, manages inventory, accepts customer orders, and coordinates with UPS to arrange package pickup and delivery.

UPS manages its truck fleet, dispatches trucks to Amazon warehouses, loads packages, and delivers them to the customer's address.

2 - Transport

1) Both Amazon and UPS MUST deploy an HTTP/1.1 server to receive requests from each other.

2) All request and response message bodies MUST be encoded in JSON format (Content-Type: application/json).

3) The hostnames and port numbers of both servers MUST be negotiated and determined before deployment and written into their respective environment configurations.

- The Amazon HTTP server MUST listen on port **8080**.
- The UPS HTTP server MUST listen on port **8081**.
- Each party MUST configure the other's hostname via environment variable (AMAZON_HOST and UPS_HOST respectively) in their docker-compose.yml.

4) Upon successful request, the receiver MUST return HTTP 200 OK. The response body format is defined in the respective interface definitions.

5) Upon request failure, the receiver MUST return HTTP 4xx. The response body MUST contain at least:
{ "error": "<error message>" }

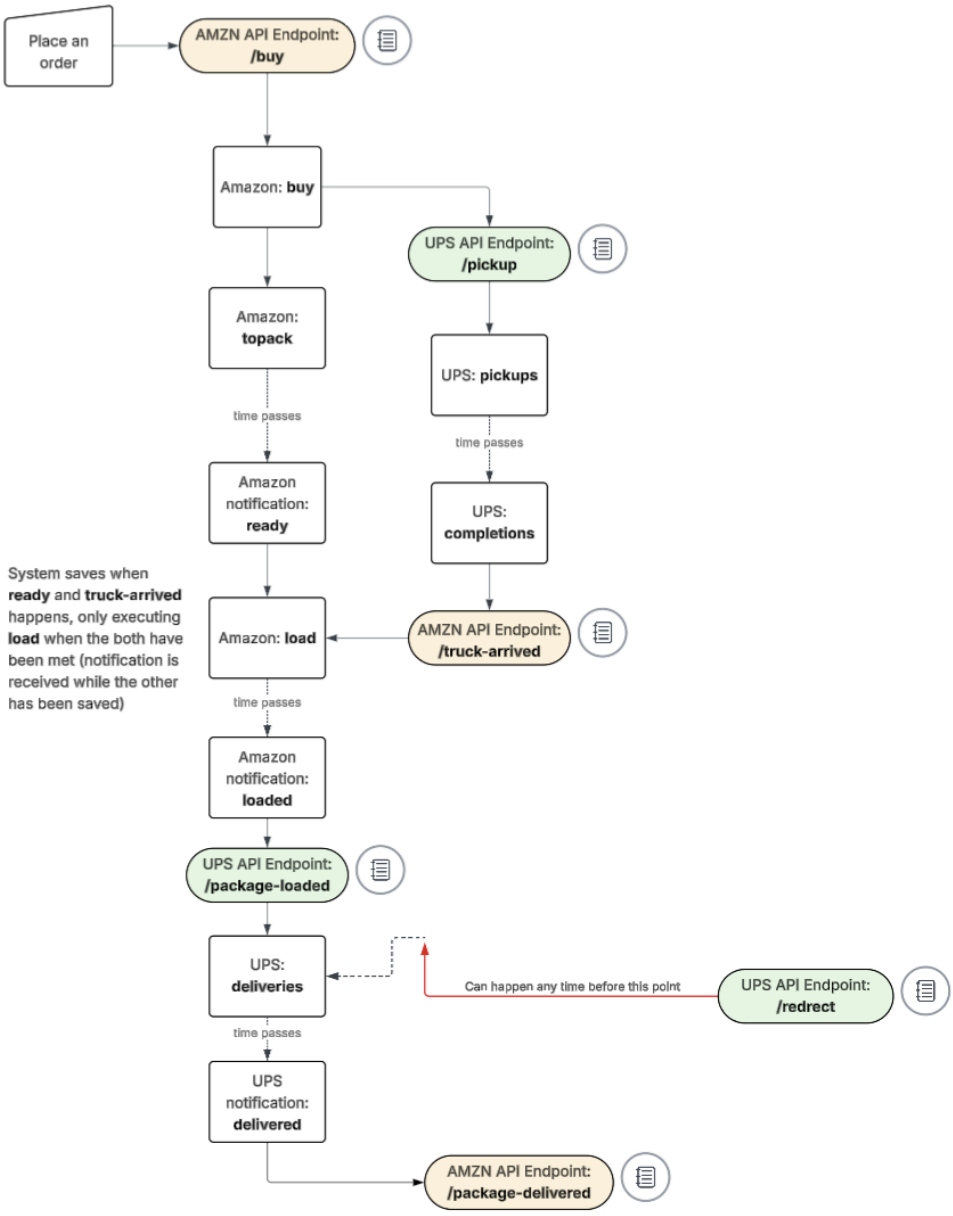
6) When a request fails due to a network error or a 5xx response, the sender SHOULD employ an exponential backoff strategy to retry (initially waiting 1 second, with a maximum of 3 retries).

3 - Interfaces

Diagram of interface usage here

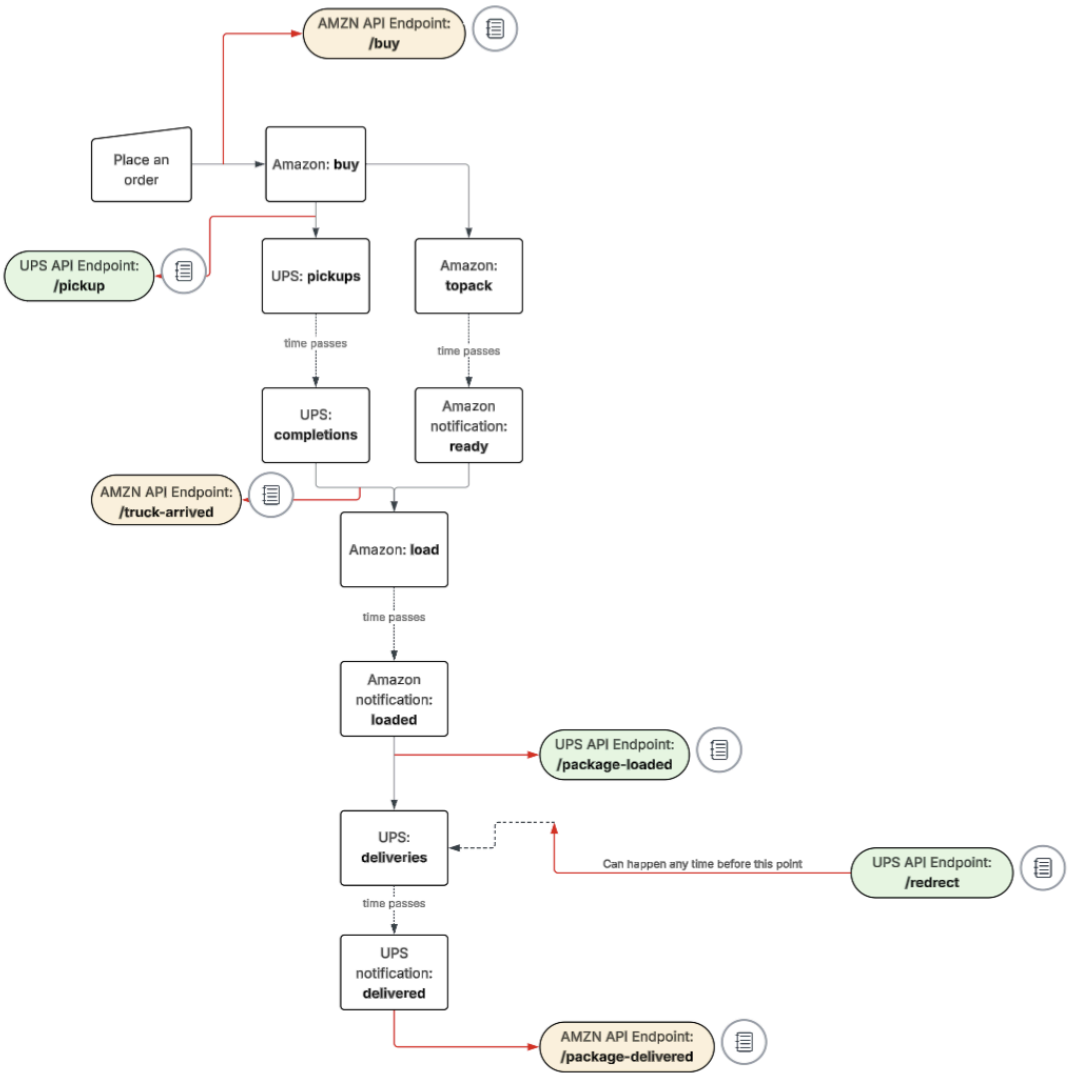
Original Diagram Modified to Include API Endpoints

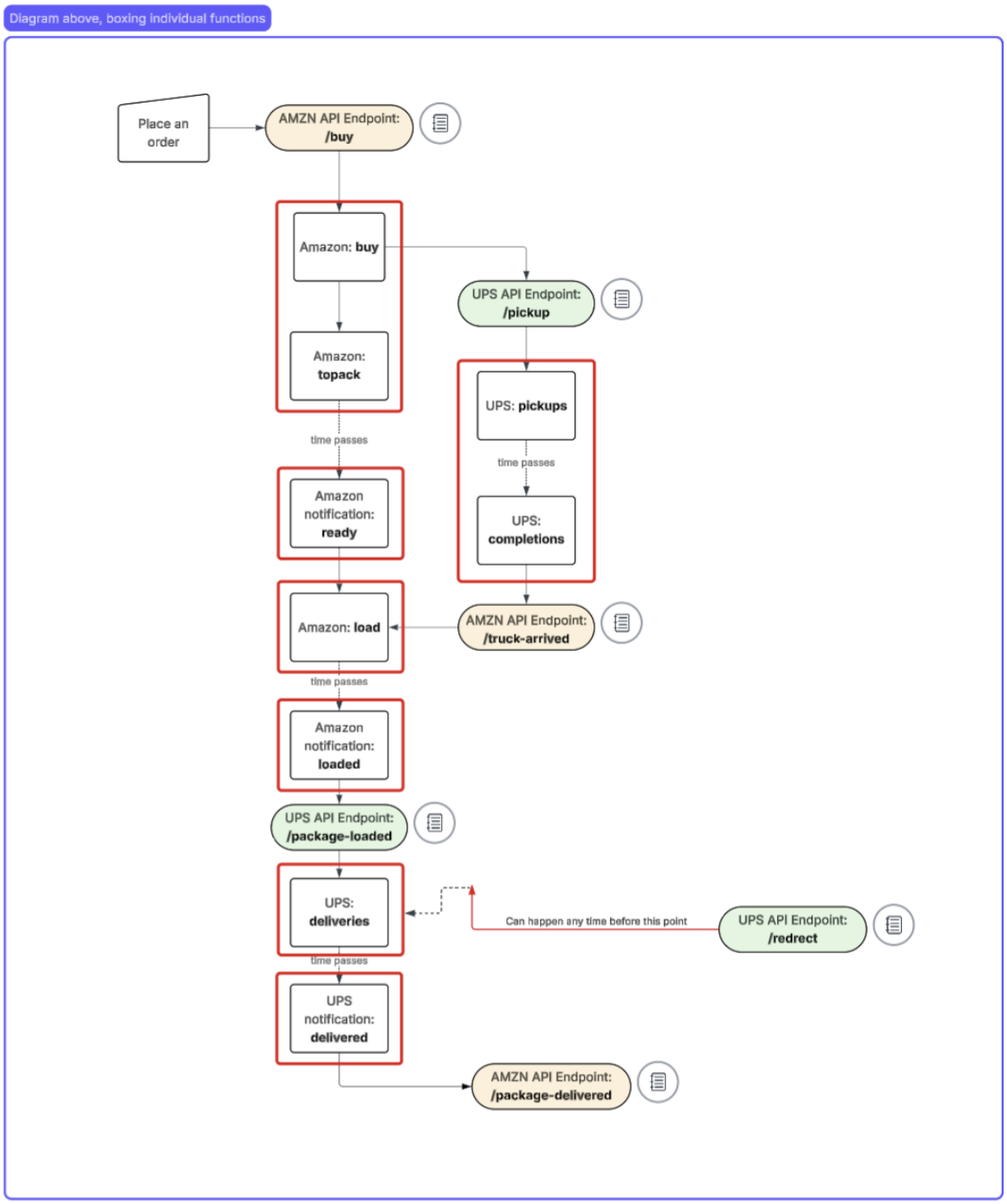
Original Diagram Modified to Include API Endpoints



Original Diagram + API Endpoint Callouts

Original diagram + API Endpoint Callouts





1) Amazon → UPS: Request Pickup

API: POST /pickup

Amazon sends the request when a user places an order and a truck needs to be dispatched to the warehouse for pickup.

Request Body:

```
{
  "package_id":   <int64>,      // MUST : A globally unique package tracking number
  "warehouse_id": <int32>,      // MUST : Warehouse ID for storing packages
  "dest_x":       <int32>,      // MUST : Delivery destination x coordinate
  "dest_y":       <int32>,      // MUST : Delivery destination y coordinate
  "ups_username": <string>      // OPTIONAL : Associated UPS user account name
}
```

Response body (200):

```
{
"truck_id": <int32> // MUST: UPS assigned truck ID
}
```

UPS MUST assign a truck and return its ID in the response. All package_id values MUST be generated by Amazon and MUST be globally unique within a given world.

<u>2) UPS → Amazon: Truck arrived at the warehouse</u>

API: POST /truck-arrived

This is sent by UPS when the specified truck arrives at the warehouse and is ready to load packages.

Request Body:

```
{
  "truck_id":     <int32>,
  "warehouse_id":       <int32>,
  "package_id":   <int64>
}
```

Response body (200):

(Empty) {}

After receiving this notification, Amazon MUST send a **load** command to World Simulator. Amazon cannot send /package-loaded until World Simulator returns a **loaded** confirmation.

3) Amazon → UPS: Package loaded, ready to depart

API: POST /package-loaded
Once World Simulator confirms that the package has been successfully loaded onto the truck, it is sent by Amazon.

Request Body:
```
{
  "package_id": <int64>,
  "truck_id":   <int32>,
  "dest_x":     <int32>,
  "dest_y":     <int32>
}
```

Response body (200):
(Empty) {}

After receiving this notification, UPS MUST send a **deliveries** instruction to World Simulator.

4) UPS → Amazon: Package delivered

API: POST /package-delivered
Once World Simulator confirms that the package has been successfully delivered, it will be sent by UPS.

Request Body:
```
{
  "package_id": <int64>
}
```

Response body (200):
(Empty) {}

After Amazon receives this notification, SHOULD will update the corresponding order status to **delivered**.

5) Amazon → UPS: Change delivery address (optional)

API: POST /redirect
This is sent by Amazon when a user requests to change the delivery address before the package arrives.

Request Body:
```
{
  "package_id": <int64>,
  "dest_x":     <int32>,
  "dest_y":     <int32>
}
```

Response body (200):
```
{
  "success": <bool>,
  "message": <string>   // OPTIONAL : eg. “The package has been dispatched.”
}
```

If the package has not yet started delivery, UPS MUST attempt to change the destination. If the package has already departed and cannot be changed, UPS MUST return **success: false** with an **explanation**. Amazon MUST **inform** the user of the result.

4 - Message Order Constraints

1) Amazon MUST NOT send **/package-loaded** before receiving **/truck-arrived** for the same **package_id**.

2) UPS MUST NOT send **deliveries** instructions to the World Simulator before receiving **/package-loaded** for the same **package_id**.

3) Amazon MUST NOT send **/pickup** repeatedly for the same **package_id** that is currently being delivered (and has not yet received **/package-delivered**).

5 - UPS User Account Association

1) If the user entered their UPS account name when placing the order, Amazon MUST include the `**ups_username**` field in the `**/pickup**` request.

2) UPS MUST bind the `**package_id**` to the corresponding `**ups_username**`, making the package appear in the user's package list.

3) If the `**ups_username**` is missing or does not exist in the UPS system, UPS SHOULD will still process the pickup request normally, treating the package as a package belonging to an unassociated user.

6 - Error handling

1) If the **/pickup** request received by UPS contains an unrecognized **warehouse_id**, a **400** error is likely.

2) If the **/truck-arrived** notification received by Amazon contains an unknown **package_id**, a **404** error is likely.

3) Both parties SHOULD **log** all errors, including the interface name, request body, and error description, for debugging.